When using my source code, make sure to run `npm install` in the extracted folder!

Useful resources:

- Pug Docs: https://pugjs.org/api/getting-started.html
- Handlebars Docs: https://handlebarsjs.com/
- EJS Docs: http://ejs.co/#docs

1️⃣ الـ Rendering Pipeline (رحلة البيانات)

الموضوع مش مجرد استبدال نصوص، ده Pipeline كامل بيمشي كالتالي:

1. **Template Parsing:** أول ما بتنادي `res.render()`، المحرك (زي EJS) بيقرأ ملف القالب من الـ Disk ويحوله لـ Memory (عشان السرعة) مخزنة في الـ **JavaScript Function** أو يترجمه لـ **Abstract Syntax Tree (AST)** (في المرات الجاية).
2. **Context Injection:** المحرك بياخد الـ JavaScript Object اللي إنت بعته (الـ Data) ويحقنه جوه الـ Scope بتاع الـ Function دي.
3. **Execution:** الـ Function بتشتغل وتجمع النصوص الثابتة مع البيانات الديناميكية.
4. **Response:** الناتج النهائي بيكون `String` عملاق هو عبارة عن الـ HTML، بيبعته السيرفر للمتصفح مع `Content-Type: text/html`.

2️⃣ الـ Core Components (المكونات الأساسية)

أي Templating Engine محترم لازم يقدم لك 3 أدوات أساسية:

A. Interpolation (الحقن)

إزاي تعرض قيمة Variable؟

- في **EJS**: بنستخدم `<% name =%>`.
- في **Pug**: بنستخدم `#{name}` أو `p= name`.

B. Control Flow (التحكم)

إزاي تعمل Loop أو Condition؟ هنا المحرك بيتحول لـ Logic Engine.

- إنت مش بس بتعرض داتا، إنت بتقرر: "لو اليوزر Admin اعرض له زرار المسح، لو لا اخفيه". ده بيحصل **قبل** ما الـ HTML يوصل للكلينت، وده قمة الأمان والـ Performance.

C. Layouts & Partials (الهندسة المعمارية)

دي أهم حتة للـ Scalability. بدل ما تكرر الـ Navbar في كل صفحة، بتعمل **Partial File** وبتربطه بالصفحات.

- **Includes:** استدعاء ملف جوه ملف.
- **Layouts/Blocks:** "إنك تعمل "برواز (Master Page) والصفحات التانية بتغير بس المحتوى اللي في النص (Content Injection).

---

3 الـ Performance & Caching

في الـ Production، المحركات دي مش بتقرأ من الـ Disk في كل Request (لأن الـ I/O مكلف).

- **Caching:** أغلب المحركات بتعمل Cache للـ "Compiled Template". مرة واحدة JS يعني بيترجم القالب لكود سريع جداً Response Time وكل اللي بيعمله بعد كدة إنه يغير الداتا بس. ده بيخلي الـ RAM، وبيركنه في الـ.

---

4 ليه ده "Old School" بس لسه "Gold"؟

رغم وجود React و Vue، الـ Templating Engines لسه وحوش في حالات معينة:

- **SEO:** لأن المحتوى بيوصل كامل للـ Crawlers بتاعة Google.
- **Speed:** مفيش JS "تقيل" بيتحمل عند الكلاينت (No Client-side hydration).
- **Security:** الـ Logic كله متداري ورا في السيرفر، اليوزر مبيشوفش غير HTML.

---

1 السطر الأول: `app.set("view engine", "pug");`

ده بنسميه **Global Configuration**. إنت هنا بتعرف الـ Default Engine.

- **إيه اللي بيحصل؟** لما بتكتب بعد كدة `res.render('shop')` من غير ما تحدد امتداد، Express لوحده بيروح يدور على ملف آخره `.pug`.
- **Built-in Integration:** مش محتاج تعمل له، Express داخلي مع Integration تقيل شوية فبيكون له Pug الـ هيعرف يندة عليه لوحده Express الـ، npm طالما إنت منزله بالـ `app.js` في ملف الـ `require`.

---

2 السطر الثاني: `app.set("views", "views");`

هنا إنت بتحدد **المكان (Location)**.

- **البارامتر الأول** `"views"`: ده اسم الإعداد (Configuration name) في Express.
- **البارامتر الثاني** `"views"`: ده اسم الفولدر اللي في الـ Root بتاع مشروعك.

- **ملحوظة للمحترفين:** لو إنت شغال من Terminal في مسار مختلف، الكود ده ممكن يضرب. الأصح والآمن إنك تستخدم الـ `path` module كدة: ده بيضمن إن Express يوصل للفولدر مهما كان المكان اللي إنت مشغل منه الـ Script.

1️⃣ الاعتماد على الـ Indentation (المسافات)

في الـ HTML العادي، أنت محتاج تقفل كل تاج (`<div/>`). في Pug، **المسافة هي اللي بتقفل التاج.**

- لما تكتب `html` وتحتها بمسافة (Indent) تكتب `head` .. المحرك بيفهم أوتوماتيك إن الـ `head` هو **Child** للـ `html` .
- أول ما ترجع بالمسافة لورا، أنت كده قفلت العنصر وبدأت عنصر جديد في نفس المستوى.

2️⃣ اختصارات العناصر والـ Classes

Pug بيستخدم سنتكس شبه **CSS Selectors**:

- `header.main-header` : ده معناه اعمل تاج `<header>` واديله `class="main-header"` .
- `nav.main-header__nav` : ده معناه اعمل تاج `<nav>` واديله Class.
- `li.main-header__item` : نفس الكلام للـ List items.

3️⃣ التعامل مع الـ Attributes (الخصائص)

الخصائص اللي زي `href` أو `src` أو `meta tags` بتتحط بين **أقواس دائرية** `()` بعد اسم العنصر مباشرة:

- `html(lang="en")` ⬅️ `<html lang="en">`
- `a(href='/')` ⬅️ `<a href="/">`

4️⃣ الـ Pipeline: إزاي الكود ده بيتحول لصفحة؟

بما إنك مفعل الـ `view engine` في Express، العملية بتمشي كالتالي:

1. **الطلب (Request):** اليوزر بيطلب صفحة الـ Home.
2. **التحليل (Parsing):** السيرفر بينادي `res.render('shop')` . محرك Pug بيمسح الكود بتاعك ويحوله لـ **JavaScript Function**.
3. **التوليد (HTML Generation):** الـ Function دي بتنتج "String" عملاق هو الـ HTML النهائي اللي فيه كل التاجات مقفولة ومظبوطة.
4. **الإرسال (Response):** السيرفر بيبعت الـ String ده للمتصفح، فالمتصفح يشوف HTML نضيف تماماً.

---

1. الـ Browser (الزبون)

ده اللي قاعد في الآخر مستني "الوجبة" (الصفحة) تجهز.

- **بيعمل إيه؟** بيبعت Request (طلب) للسيرفر يقوله: "هات لي صفحة الـ Shop".
- **مبيعملش إيه؟** مبيفهمش يعني إيه **Pug**! هو مبيشوفش غير HTML نضيف.

2. الـ Express.js (المقاول/الـ Router)

ده اللي بياخد الطلب من الزبون ويقرر هيروح لمين.

- **بيعمل إيه؟** بيشوف المسار (`/` أو `admin/`) ويروح للـ Function الصح.
- **دوره في الـ Pug:** هو اللي بينادي على "محرك القوالب" ويقوله: "خد الداتا دي (المنتجات) وخد الملف ده (`shop.pug`) واطبخهم مع بعض".

3. محرك الـ Pug (الشيف/المطبخ)

ده الصنايعي اللي إنت بتكتبله الكود المختصر.

- **بيعمل إيه؟** بياخد ملف الـ `.pug` اللي إنت كاتبه، ويفك "الشفرة" بتاعته (يزود التاجات اللي ناقصة، يقفل الـ `<div/>` اللي إنت مش كاتبها)، ويحط الـ Variables في مكانها.
- **الناتج بتاعه:** بيطلع **HTML String** كامل ويديه لـ Express عشان يبعته للبراوزر.

تعال نشوفها في الكود (المثال اللي إنت كاتبه):

إنت كتبت في الـ **Pug** ⬅️ `li.main-header__item` :Pug بيشوفها ويقول: "أها، هو يقصد `<li>` وليها Class".

إنت كتبت في الـ `app.set("view engine", "pug")` :Express ⬅️ إنت هنا بتقول لـ **Express**: "لو سمحت لما أقولك `render` استخدم الشيف اللي اسمه **Pug** هو اللي فاهم اللغة دي".

1️⃣ الـ Compilation Caching (الترجمة لمرة واحدة)

محرك Pug مش "غبي" عشان يترجم الملف من الصفر في كل مرة يوزر يطلب الصفحة.

- **اللي بيحصل:** أول مرة الصفحة بتطلب، Pug بيحول الكود لـ **JavaScript Function**.
- **المرة التانية:** بينفذ الـ Function دي فوراً في الـ Memory. ده بيخلي وقت التجهيز (Processing Time) يقاس بالـ **Milliseconds** (أجزاء من الألف من الثانية).

2️⃣ الـ I/O Bottleneck (المشكلة الحقيقية)

أبطأ حاجة في السيرفر هي إنه يروح يقرأ ملف من الـ Hard Disk (I/O Operation).

- سواء كان الملف `index.html` أو `index.pug` السيرفر كدة كدة هيعمل I/O.
- بما إن ملف الـ **Pug** حجمه أصغر بكتير (نص حجم الـ HTML تقريباً)، فقراءته من الديسك أحياناً بتكون أسرع!

3️⃣ الـ Network Transfer (السرعة عند اليوزر)

في الحالتين (HTML أو Pug)، اللي بيمشي في السلك وبيروح للمتصفح هو **HTML**. فمن وجهة نظر اليوزر، مفيش فرق في سرعة التحميل لأن حجم البيانات اللي واصلة للمتصفح واحدة.

🧠 الفكرة الأساسية: الـ "Skeleton" والـ "Holes"

الـ Layout هو عبارة عن هيكل عظمي فيه "خروم" أو "فتحات" (اللي هي الـ `block`). الصفحات اللي بتعمل `extends` للـ Layout ده، وظيفتها بس إنها "تملى الخروم" دي بالكود بتاعها.

1️⃣ الـ `block styles` : الفتحة المرنة

إنت حطيت `block styles` تحت الـ `main.css`.

- **ليه دي حركة ذكية؟** لأنك كدة بتقول: "كل الصفحات هتاخد الـ `main.css` غصب عنها، بس لو في صفحة محتاجة CSS زيادة (زي صفحة الـ Form)، تقدر تحشره في الفتحة دي".

2️⃣ الـ `block content` : قلب الصفحة

دي أهم فتحة عندك. دي اللي هيتحط فيها الـ Grid بتاع المنتجات في صفحة الـ Shop، أو الـ Form في صفحة الـ Add Product.

3️⃣ الـ Dynamic Classes: ذكاء الـ Navbar

السطر ده عبقري: `class = (path === '/' ? 'active' : '')` إنت هنا مش كاتب كود ثابت. إنت بتقول لـ Pug: "شوف الـ `path` اللي جاي لك من السيرفر، لو هو `/` حط class اسمه `active` .. لو مش هو، سيبها فاضية". كدة الـ Navbar هينور عند الصفحة اللي اليوزر واقف فيها لوحده.

🛠️ إزاي نستخدم الـ Layout ده؟ (Deep Dive in Implementation)

تخيل إن الملف اللي إنت كاتبه ده اسمه `main-layout.pug` وموجود في فولدر اسمه `layouts`.

صفحة الـ (`shop.pug`) Shop:

Code snippet

```
extends layouts/main-layout.pug

block styles
    link(rel="stylesheet", href="/css/product.css")

block content
    main
        if prods.length > 0
            .grid
                each product in prods
                    // كود الكارت هنا...
        else
            h1 No Products Found!
```

🔍 ليه الـ Layouts بتوفر وقتك؟ (The Technical Advantage)

1. **Maintenance:** هتغيرها في ملف واحد بس، وهتسمع في الـ `lang="ar"` ، وتخليها `lang="en"` لو حبيت تغير الـ 50 صفحة بتوع مشروعك.
2. **Clean Controllers:** السيرفر دلوقتي بيبعت بس `pageTitle` و `path` و `prods` .. وموش شاغل باله الـ HTML هو اللي شايل الليلة Layout شكله عامل إزاي، الـ.
3. **No Code Repetition (DRY):** بطلنا نكرر الـ Header والـ Nav أصغر 80% في كل ملف. كودك بقى.

- `app.engine("hbs", ...)` : وهو "اسم حركي" إنت بتديله الـ `hbs` . إنت هنا مش بس بتفعل المحرك، الـ Express روح نادى على مكتبة `express-handlebars` لما تلاقي ملفات آخره `.hbs` ، Express لوحده، فإنت بتقوله: "يا HBS مبيفهمش هي اللي هتتصرف".
- `app.set("view engine", "hbs")` : بتقول للـ "الكلمة الأخيرة". دي `res.render` إن المحرك الافتراضي في كل مرة، اكتب `res.render('shop.hbs')` كدة مش محتاج تكتب `hbs` . بتاعنا هو الـ `res.render('shop')` وهو هيفهم.

3️⃣ الـ Deep Dive: ليه HBS بيحتاج `app.engine` ؟

عكس **Pug**، الـ Handlebars فيه إمكانيات كتير (زي الـ Layouts والـ Helpers) مش Built-in جوه Express. عشان كدة بنستخدم `app.engine` عشان نبعت "إعدادات" (Configurations) للمحرك ده، زي:

- إيه هو الـ **Default Layout**؟
- هل في **Partials** (قطع كود صغيرة زي الـ Footer)؟
- إيه هو الـ **Extension** اللي هنستخدمه؟

1️⃣ الـ `{{#if}}` (البواب)

ده المسؤول عن "الشرط". في HBS، الـ `if` مبيفهمش غير (True أو False).

- **بيعمل إيه؟** لو المتغير اللي بعده موجود وفيه داتا، بيعرض اللي جواه. لو فاضي أو `false` بيفوته.
- **مبيعملش إيه؟** مبيعملش عمليات حسابية (يعني مفيش `x > 5` جوه الـ HBS). لازم تبعت نتيجة الشرط جاهزة من الـ Express.

2️⃣ الـ `{{#each}}` (الونش)

ده اللي بيلف على الـ **Arrays**.

- أول ما بتقوله `{{#each products}}` .. هو بيمسك الـ Array ده بيفكه، وبيكرر كل الـ HTML اللي بينه وبين القفلة بتاعته بعدد العناصر اللي في الـ Array.
- جوه الـ `each` .. إنت بقيت "جوه" العنصر نفسه، فممكن تطلب خصائصه مباشرة.

3️⃣ الـ Keyword `this` (صاحب البيت)

كلمة `this` في Handlebars معناها **"أنا واقف فين دلوقتي؟"**.

- لما تكون جوه `{{#each}}` .. الـ `this` بتشاور على "المنتج الحالي" اللي الونش شايله دلوقتي.
- لو المنتج عبارة عن String بس (مش Object)، بتكتب `{{this}}` عشان تعرضه.
- لو المنتج Object، بتكتب `{{this.title}}` .. وممكن تختصرها وتكتب `{{title}}` علطول، الـ HBS ذكي وهيفهم إنك تقصد `title` بتاع العنصر اللي إنت واقف عنده.

4️⃣ الـ Closing Tags (القفلة)

في الـ HBS، أي "بلوك" بيبدأ بـ `#` لازم ينتهي بـ `/` .

- بدأت بـ `{{#if}}` ⬅️ تقفل بـ `{{/if}}` .
- بدأت بـ `{{#each}}` ⬅️ تقفل بـ `{{/each}}` .

- لو نسيت القفلة، الـ Engine هيطلع لك Error يقولك "Unexpected end of file" لأنه لسه مستني بقية الـ "بلوك".

---

1 الـ `app.engine("hbs", ...)`

هنا إنت مش بس بتشغل مكتبة، إنت بتعمل "Contract" (عقد) مع Express:

- **"hbs"**: ده الاسم الحركي (Alias). إنت بتقول لـ Express لما أستخدم كلمة "hbs" في أي مكان، روح نادى على الـ Package اللي اسمها `express-handlebars`.
- ممكن تسميها "shark" لو عايز، بس ساعتها ملفاتك لازم تنتهي بـ `.shark`. فطبعاً "hbs" هي المعيار.

---

2 الـ `"/layoutDir: "views/layouts`

دي "الخريطة" اللي السيرفر بيمشي عليها.

- بيحب النظام. إنت هنا بتقوله: "يا معلم، لما أقولك فيه Handlebars لوحده، لكن Layouts كان بيعرف مكان الـ Pug اللي جواه `layouts` الرئيسي، ادخل جوه فولدر `views` متدورش في فولدر Layout".
- **ليه بنعمل كدة؟** عشان ملفات الـ Logic (زي الـ shop والـ add-product) متتمسحش في وسط ملفات الـ Layout، فالدنيا تبقى منظمة.

---

3 الـ `"defaultLayout: "main-layout`

دي "الجوكر" بتاعك، وأهم سطر في الإعدادات.

- من غير السطر ده، كنت هتضطر في كل `res.render` تكتب: `res.render('shop', {layout: 'main-layout'})`.
- بالسطر ده، إنت بتقوله: "أي ملف أعمله Render، روح البسه البرواز اللي اسمه `main-layout.hbs` أوتوماتيك، إلا لو أنا قولتلك غير كدة".

---

4 الـ `"extname: "hbs`

دي "مقص القماش".

- في الأصل بيدور على ملفات امتدادها طويل ورخم `.handlebars`. Handlebars
- بما إننا مبرمجين وبنحب نختصر، السطر ده بيخلي المحرك يدور على `.hbs` بس. وده بيوفر عليك كتابة حروف كتير وبيخلي شكل الملفات أنضف.

🧠 الـ Logic الـ Deep بجد (ازاي الـ Layout بيشتغل؟)

لما إنت بتطلب صفحة الـ Shop، الـ Engine بيعمل الخطوات دي بالترتيب:

1. بيروح يفتح `main-layout.hbs` .
2. بيفضل يقرأ الـ HTML لحد ما يخبط في كلمة `{{{ body }}}` .
3. يوقف قراية، ويروح يفتح ملف `shop.hbs` .
4. ياخد كل اللي جوه `shop.hbs` ويحشره مكان الـ `{{{ body }}}` .
5. لو فيه `{{if#}}` أو `{{each#}}` جوه الـ `shop.hbs` بيحلهم الأول بالداتا اللي إنت بعتها من الـ Router، وبعدين يحط الناتج النهائي في الـ Layout.

⚠️ تريكة الـ Triple Stash `{{{ }}}`

إنت أكيد لاحظت إننا بنستخدم `{{ pageTitle }}` (قوسين) وبنستخدم `{{{ body }}}` (تلاتة).

- **القوسين `{{ }}` :** بيعملوا حاجة اسمها "Escape". يعني لو المتغير جواه تاجات HTML (زي `<b>Text</b>`) هيظهرها للمستخدم كنص عادي مش خط تقيل. وده للأمان (Security) عشان محدش يحقن كود Script في موقعك.
- **التلاتة `{{{ }}}` :** بتقول للمحرك: "عدي الكود ده زي ما هو، ده HTML حقيقي وأنا واثق فيه". وعشان كدة الـ `body` لازم ياخد 3 أقواس عشان التاجات اللي جواه تشتغل.

1️⃣ الفرق بين الـ Tags في EJS

الـ EJS بيعتمد على تلات أنواع من التاجات (السيوف) اللي بتتحكم في الكود:

- `<% logic %>` : دي بنسميها **Control Flow**. والـ `if` بنستخدمها للأوامر اللي "مش بتطلع كلام" في الصفحة، زي الـ `while` والـ `for` . بس Logic أي حاجة بتعمل.
- `<%= variable %>` : دي **Output Tag**. زي اسم المنتج أو سعر (HTML بتستخدمها عشان "تطبع" قيمة متغير جوه الـ الكتاب). دي بتعمل Escape للـ HTML (أمان).
- `<%- include(...) %>` : دي بتستخدمها لو عايز تطبع "خام" HTML (Raw) زي ما هو، أو تستدعي ملف تاني (Partial).

2️⃣ الـ Deep Dive في الكود بتاعك

إنت عملت حاجتين Handlebars مكنش بيعرف يعملهم بسهولة:

- **تشيك الـ Length مباشرة:** `<% } if(products.length > 0) %>` هنا إنت مش محتاج تبعت `hasProducts: true` من السيرفر. إنت سألت الـ Array نفسه: "إنت فيك داتا؟". دي حرية الـ EJS.
- **الـ For Loop الحقيقية:** `<% } for (let product of products) %>` إنت هنا بتستخدم `for...of` بتاعة الـ JavaScript العادية خالص. وده بيخليك تقدر تعمل Logic معقد جوه الـ Loop لو حبيت (زي إنك تطلع Index مثلاً).

3️⃣ الـ Closing Tags (المتاهة)

أكتر حاجة بتلخبط في EJS هي قفلة القوس `<% {}` . لاحظ إنك لازم تقفل كل `if` وكل `for` بـ `<%{}%>` في المكان الصح، وإلا الـ Engine هيتوه ويطلع لك Error إن فيه "Unexpected identifier".

4️⃣ الـ Layouts في EJS (مختلفة تماماً)

خلي بالك، الـ EJS "مبيفهمش" فكرة الـ `main-layout.hbs` والـ `{{{body}}}` أوتوماتيك. في EJS بنستخدم فلسفة الـ **Includes**: إنت بتقسم الصفحة لـ `header.ejs` و `footer.ejs` و `head.ejs` .. وفي كل صفحة (زي الـ shop) بتنادي عليهم كدة:

Code snippet

```
<%- include('includes/head.ejs') %>
</head>
<body>
    <%- include('includes/navigation.ejs') %>
    <main>
        </main>
<%- include('includes/end.ejs') %>
```

1️⃣ فلسفة الـ `include` (قطاع الغيار)

في EJS، إحنا مش بنستخدم "برواز" واحد (Layout) زي الـ Handlebars، إحنا بنستخدم فكرة **"المكعبات"**.

- إنت بتقسم صفحة الـ HTML لقطع صغيرة (Partials).
- كل قطعة بتشيل وظيفة معينة (مثلاً ملف للـ `<head>` , ملف للـ `<nav>` , ملف للـ `<footer>`).
- **السر في التاج `<%-`** : لاحظ إننا استخدمنا علامة الناقص (-) مش اليساوي (=). ليه؟ عشان الـ `include` ده جواه كود HTML، وإحنا عايزين الـ EJS "يفكه" ويرسمه كـ HTML حقيقي مش مجرد نص.

2️⃣ شرح الملفات اللي إنت ناديتها (The Structure)

إنت مقسم الكود لـ 3 محطات أساسية:

🟢 المحطة الأولى: `includes/head.ejs`

دي بتبدأ من أول `<DOCTYPE html!>` لحد قفلة الـ `<head/>`.

- **الفائدة:** لو حبيت تضيف مكتبة CSS جديدة (زي Bootstrap مثلاً)، هتضيفها في ملف واحد بس، وهتسمع في كل صفحات الموقع "أوتوماتيك".

🔵 المحطة الثانية: `includes/navigation.ejs`

دي اللي فيها الـ `<header>` والـ `<nav>`.

- **التحدي:** إزاي الـ Navigation بيعرف ينور الزرار الصح (Active class)؟
- في EJS بنعمل كدة: `<a class="<%= path === '/' ? 'active' : '' %>" href="/">Shop</a>`. بنستخدم الـ Logic بتاع الـ JS جوه التاج.

🔴 المحطة الثالثة: `includes/end.ejs`

دي اللي فيها قفلة الـ `<body/>` والـ `<html/>` وأي ملفات (`<script>`) JavaScript.

3️⃣ إزاي الـ EJS بيجمع "الطبخة" دي؟

لما بتطلب صفحة الـ 404، الـ Engine بيعمل الآتي:

1. بيفتح ملف الـ `ejs.404`.
2. بيلاقي أول سطر `include('includes/head')` ⬅️ يروح يجيب كود الـ Head ويحطه مكانه.
3. بيلاقي السطر التاني `include('includes/navigation')` ⬅️ يجيب كود الـ Nav ويحقنه.
4. بيكمل قراية الـ HTML العادي اللي في النص (`<h1>Page Not Found!</h1>`).
5. وفي الآخر يقفل بملف الـ `end`.

4️⃣ الـ Deep Dive: ليه EJS معندوش `main-layout` زي HBS؟

بص يا هندسة، الـ EJS سياسته "الحرية المطلقة".

- **Handlebars** بيمشي بنظام **Top-Down**: (الـ Layout هو اللي بينادي على الـ Body).
- **EJS** بيمشي بنظام **Bottom-Up**: (الصفحة هي اللي بتنادي على الأجزاء اللي محتاجاها).

الميزة هنا: إنك ممكن تعمل صفحة بـ Navigation مختلف تماماً عن بقية الموقع بسهولة، بمجرد إنك تغير الـ `include` لملف تاني. في Handlebars الموضوع ده كان بيبقى "لف ودوران" كتير.

💡 تريكة الـ "Relative Paths":

خلي بالك وإنت بتنادي الـ `include` .. الـ EJS بيبدأ يدور من فولدر الـ `views` .

- لو ملف الـ `ejs.404` موجود في الـ root بتاع `views` .
- وملف الـ `head` موجود جوه فولدر `includes` .
- بتكتب المسار كدة: `includes/head` (ومش لازم تكتب `ejs.` لأن المحرك عارف الامتداد).